54. Redis Cache: `stores` and `ttl` Explained (with an API Example)

Continues from **theory #49 (Redis basics)**. This page explains these two lines in `app.module.ts`:

```
CacheModule.registerAsync({
  isGlobal: true,
  useFactory: () => ({
    stores: [new KeyvRedis('redis://localhost:6379')],   // ← WHERE to save
    ttl: 60_000,                                         // ← HOW LONG to keep
  }),
}),
```

> **In one line:** "Store the cache in Redis on my laptop, and delete each item automatically after 1 minute."

1. `stores: [new KeyvRedis('redis://localhost:6379')]`

This means **"save the cache in Redis, running on my laptop at port 6379."**

Part	Meaning
`stores`	**Where** cached data is kept
`KeyvRedis`	A connector that lets NestJS talk to Redis
`redis://`	The protocol: "this is a Redis server"
`localhost`	On **this** computer (your Docker Redis)
`6379`	Redis's default port

Without Redis vs with Redis

Without `stores` (default: in-memory)	With Redis
The cache is kept inside the NestJS app's own memory	The cache is kept in a separate Redis server
Restart `npm run start:dev` and the cache is **wiped** ✗	Restart the app and the cache is **still there** ✓
2–3 copies of the app each have their **own** cache	All copies **share one** cache

> **Real-life example:** Without Redis, each shop worker keeps notes in their own pocket, and the notes are lost when they go home. With Redis, everyone writes on **one shared notice board** that stays on the wall.

2. `ttl: 60_000`

TTL = Time To Live: how long a saved item stays before it is **deleted automatically**.

- `60_000` is just **60000**. The `_` only makes it easier to read (like writing 60,000).
- The unit is **milliseconds**, so 60000 ms = **60 seconds = 1 minute**.

Value	Time
`10_000`	10 seconds
`60_000`	1 minute
`5 * 60_000`	5 minutes
`60 * 60_000`	1 hour

> **Real-life example: milk** 🥛 Milk has an expiry date. After it expires, you throw it away and buy fresh milk. Cached data is the same: after 60 seconds it "expires", Redis throws it away, and the next request fetches fresh data from the database.

> **Units differ!** In NestJS cache-manager, TTL is in **milliseconds** (`60_000`). In `redis-cli`, `SET key value EX 60` uses **seconds**.

3. API example

student.controller.ts

```
@Get(':id')
findOne(@Param('id', ParseIntPipe) id: number) {
  return this.studentService.findOne(id);
}

@Patch(':id')
update(@Param('id', ParseIntPipe) id: number, @Body() dto: UpdateStudentDto) {
  return this.studentService.update(id, dto);
}
```

student.service.ts

```
@Injectable()
export class StudentService {
  constructor(
    @InjectRepository(Student) private repo: Repository<Student>,
    @Inject(CACHE_MANAGER) private cache: Cache,
  ) {}

  async findOne(id: number) {
    const key = `student:${id}`;

    const cached = await this.cache.get(key);
    if (cached) {
      console.log('⚡ from REDIS');
      return cached;
    }

    console.log('🐢 from POSTGRES');
    const student = await this.repo.findOneBy({ id });
    await this.cache.set(key, student);   // expires in 60 sec (default ttl)
    return student;
  }

  async update(id: number, dto: UpdateStudentDto) {
    await this.repo.update(id, dto);
    await this.cache.del(`student:${id}`);   // remove old copy now
    return this.findOne(id);
  }
}
```

4. Cache expires after 60 seconds: timeline

Call `GET http://localhost:3000/students/1` in Postman several times:

Time	Request	Console says	Why
10:00:00	1st call	🐢 from POSTGRES	Redis is empty → get from DB → save in Redis
10:00:10	2nd call	⚡ from REDIS	Saved 10 sec ago
10:00:45	3rd call	⚡ from REDIS	Still within 60 sec
10:01:00	—	—	⏰ **Redis deletes `student:1` automatically**
10:01:20	4th call	🐢 from POSTGRES	Expired → fetch again → saved for another 60 sec
10:01:30	5th call	⚡ from REDIS	Fresh copy



> The **API keeps working** after the cache expires. The response is **the same data every time**; only **where it comes from** changes. When the data isn't in Redis, it comes from Postgres and is cached again.

See it in Redis yourself

```
docker exec -it redis redis-cli

KEYS *                # → "keyv:student:1"  (KeyvRedis may add a prefix)
TTL keyv:student:1    # → 42   (seconds left; use the exact name KEYS shows)
# wait 1 minute...
KEYS *                # → (empty array), auto-deleted ✅
```

5. ⚠ The catch: updates within the 60 seconds

If you **don't** delete the cache when updating:

Time	Request	Result
10:00:00	`GET /students/1`	`{ name: "Gourish" }` (saved in Redis)
10:00:20	`PATCH /students/1`	Name changed to "Gourish Naik" in Postgres
10:00:30	`GET /students/1`	`{ name: "Gourish" }` ❌ **OLD data from Redis!**
10:01:00	—	Cache expires
10:01:05	`GET /students/1`	`{ name: "Gourish Naik" }` ✅ finally correct

For up to 60 seconds, users see **stale (old) data**.

Fix: delete the key when the data changes

```
async update(id: number, dto: UpdateStudentDto) {
  await this.repo.update(id, dto);
  await this.cache.del(`student:${id}`);   // 🗑 remove old copy NOW
}

async remove(id: number) {
  await this.repo.delete(id);
  await this.cache.del(`student:${id}`);
}
```

Time	Request	Result
10:00:00	`GET /students/1`	`{ name: "Gourish" }` (saved in Redis)
10:00:20	`PATCH /students/1`	Postgres updated + **Redis key deleted**
10:00:30	`GET /students/1`	`{ name: "Gourish Naik" }` ✅ fresh from Postgres, cached again

6. How long should the TTL be?

TTL	Result
Too short (1 sec)	The cache barely helps; you hit the database almost every time
Too long (1 day)	Users may see old data after changes (if you forget `del`)
1 minute	A safe default for most data ✅

You can override the default for **one item**:

```
await this.cache.set('settings', data, 60 * 60_000);   // this one: 1 hour
await this.cache.set(`otp:${phone}`, otp, 5 * 60_000);   // OTP: 5 minutes
```

Kind of data	Suggested TTL
App settings, country list, categories (rarely change)	1 hour – 1 day
Product list, student profile	1 – 10 minutes
OTP / reset token	5 – 15 minutes
Live prices, stock count, bank balance	Seconds, or don't cache at all

7. Quick summary

Question	Answer
What does `stores` do?	Decides **where** the cache lives (here: Redis at `localhost:6379`)
Why Redis and not the default?	It survives app restarts and is shared by all app copies
What is `ttl`?	Time To Live: how long before an item is auto-deleted
`60_000` means?	60000 milliseconds = **60 seconds**
After 60 sec?	Redis deletes the item. The next request gets it from the DB and caches it again.
Does the API break when the cache expires?	No. It's just a bit slower for that one request.
Biggest mistake	Not calling `cache.del()` after update/delete, so users see stale data